



Métodos Numéricos Computacionais

Igor dos Reis Gomes

RELATÓRIO TRABALHO 5 – AJUSTE DE CURVAS

Métodos Numéricos Computacionais

Bauru

2025

Bauru
2025
Igor dos Reis Gomes

RELATÓRIO TRABALHO 5 – AJUSTE DE CURVAS

Métodos Numéricos Computacionais

Trabalho teórico-prático apresentado como parte da composição da nota da disciplina de Métodos Numéricos Computacionais – Prof. Douglas Rodrigues.

1. OBJETIVO

O objetivo deste trabalho é aplicar e comparar os métodos de ajuste de curvas que foram introduzidos na disciplina de Métodos Numéricos Computacionais, com base em dados experimentais de crescimento populacional ao longo do tempo, que foram dados no enunciado do trabalho. Foram implementados os ajustes linear, polinomial (graus 2, 3 e 4) e o exponencial, utilizando o método dos mínimos quadrados. A análise também inclui a geração dos erros médios quadráticos e os coeficientes de determinação (R^2), com o objetivo de verificar a qualidade e precisão de cada modelo.

2. EXPLICAÇÃO DO CÓDIGO

Neste trabalho, o código foi desenvolvido utilizando a linguagem de programação Python, buscando resolver o problema proposto. Assim, o programa foi organizado de forma modular, com funções responsáveis por cada parte do processo: cálculos matemáticos, ajustes de curvas, métricas de erro, geração de gráficos e interface de execução no terminal.

2.1. Ajuste Linear

A função `ajuste_linear()` aplica o método dos mínimos quadrados para ajustar uma reta $y = ax + b$. O sistema é resolvido por meio de uma matriz A e um vetor b , construídos com base nas somas dos valores de x , y , x^2 e $x \cdot y$. Segue abaixo o trecho do código:

Trecho 1 - Implementação do ajuste linear

```
1 def ajuste_linear(x, y):
2     A = np.array([[len(x), acumulo(x)], [acumulo(x), acumuloPotencia(x, 2)]])
3
4     b = np.array([acumulo(y), produtoSimples(x, y)])
5
6     coef = np.linalg.solve(A, b)
7
8     return coef[1], coef[0]
```

Fonte: Elaboração Própria

2.2. Ajustes Polinomiais de Graus 2, 3 e 4

As funções `ajuste_poli2()`, `ajuste_poli3()` e `ajuste_poli4()` utilizam o mesmo princípio do ajuste linear, mas aumentam a matriz do sistema para incluir potências maiores de x até o grau necessário. Os coeficientes do polinômio são obtidos por meio da resolução do sistema linear. Abaixo mostra os trechos para cada ajuste polinomial implementado:

Trecho 2 - Implementação do ajuste polinomial de grau 2

```
1 def ajuste_poli2(x, y):
2     A = np.array([
3         [len(x), acumulo(x), acumuloPotencia(x, 2)],
4         [acumulo(x), acumuloPotencia(x, 2), acumuloPotencia(x, 3)],
5         [acumuloPotencia(x, 2), acumuloPotencia(x, 3), acumuloPotencia(x, 4)]
6     ])
7
8     b = np.array([acumulo(y), produtoSimples(x, y), produtoPotencia(x, y, 2)])
9
10    coef = np.linalg.solve(A, b)
11
12    return coef[:-1]
```

Fonte: Elaboração Própria

Trecho 3 - Implementação do ajuste polinomial de grau 3

```
1 def ajuste_poli3(x, y):
2     A = np.array([
3         [len(x), acumulo(x), acumuloPotencia(x, 2), acumuloPotencia(x, 3)],
4         [acumulo(x), acumuloPotencia(x, 2), acumuloPotencia(x, 3), acumuloPotencia(x, 4)],
5         [acumuloPotencia(x, 2), acumuloPotencia(x, 3), acumuloPotencia(x, 4), acumuloPotencia(x, 5)],
6         [acumuloPotencia(x, 3), acumuloPotencia(x, 4), acumuloPotencia(x, 5), acumuloPotencia(x, 6)]
7     ])
8
9     b = np.array([acumulo(y), produtoSimples(x, y), produtoPotencia(x, y, 2), produtoPotencia(x, y, 3)])
10
11    coef = np.linalg.solve(A, b)
12
13    return coef[:-1]
```

Fonte: Elaboração Própria

Trecho 4 - Implementação do ajuste polinomial de grau 4

```
1 def ajuste_poli4(x, y):
2     A = np.array([
3         [len(x), acumulo(x), acumuloPotencia(x, 2), acumuloPotencia(x, 3), acumuloPotencia(x, 4)],
4         [acumulo(x), acumuloPotencia(x, 2), acumuloPotencia(x, 3), acumuloPotencia(x, 4), acumuloPotencia(x, 5)],
5         [acumuloPotencia(x, 2), acumuloPotencia(x, 3), acumuloPotencia(x, 4), acumuloPotencia(x, 5), acumuloPotencia(x, 6)],
6         [acumuloPotencia(x, 3), acumuloPotencia(x, 4), acumuloPotencia(x, 5), acumuloPotencia(x, 6), acumuloPotencia(x, 7)],
7         [acumuloPotencia(x, 4), acumuloPotencia(x, 5), acumuloPotencia(x, 6), acumuloPotencia(x, 7), acumuloPotencia(x, 8)]
8     ])
9     b = np.array([
10         acumulo(y), produtoSimples(x, y), produtoPotencia(x, y, 2), produtoPotencia(x, y, 3), produtoPotencia(x, y, 4)
11     ])
12     coef = np.linalg.solve(A, b)
13     return coef[:-1]
```

Fonte: Elaboração Própria

2.3. Ajuste Exponencial

A função `ajuste_exponencial()` utiliza o modelo na forma $y = ae^{bx}$. Para linearizar esse modelo, aplica-se o logaritmo natural aos valores de y , transformando-o em uma equação linear que pode ser resolvida com o método dos mínimos quadrados.

Trecho 5 - Implementação do ajuste exponencial

```
1 def ajuste_exponencial(x, y):
2     soma_ln_y, ln_y = logaritmo_natural(y)
3     A = np.array([[produtoSimples(x, x), acumulo(x)], [acumulo(x), len(x)]])
4     b = np.array([produtoSimples(x, ln_y), soma_ln_y])
5     sol = np.linalg.solve(A, b)
6     a = e**sol[1]
7     b = sol[0]
8     return a, b
```

Fonte: Elaboração Própria

2.4. Cálculo dos Erros e R^2

As funções auxiliares `erro_total()`, `erro_residual()`, `erro_medio()` e `r2()` são utilizadas para calcular os erros quadráticos médios (EQM) e os coeficientes de determinação (R^2):

Trecho 6 - Implementação do EQM e R^2

```
1 def erro_total(y): return sum((yi - np.mean(y))**2 for yi in y)
2 def erro_residual(y, y_calc): return sum((yi - yj)**2 for yi, yj in zip(y, y_calc))
3 def erro_medio(sqr, n): return sqrt(sqr / n)
4 def r2(sqr, y): return 1 - sqr / y
```

Fonte: Elaboração Própria

O EQM mede o desvio médio entre os valores reais e os ajustados, indicando o quão próximo o modelo está dos dados. Já o R^2 mostra o quanto da variação dos dados é explicada pelo modelo, sendo melhor quanto mais próximo de 1. Essas métricas são aplicadas a todos os modelos ajustados.

2.5. Geração dos Gráficos

Trecho 7 - Implementação dos Gráficos dos Ajustes de Curva

```
1 def gerar_graficos_ajustes(x, y):
2     modelos = ['Linear', 'Polinomial 2º', 'Polinomial 3º', 'Polinomial 4º', 'Exponencial']
3     nomes_arquivos = [
4         'ajuste_linear.png',
5         'ajuste_poli2.png',
6         'ajuste_poli3.png',
7         'ajuste_poli4.png',
8         'ajuste_exp.png'
9     ]
10
11     x_continuo = np.linspace(min(x), max(x), 200)
12
13     a1, b1 = ajuste_linear(x, y)
14     y1_continua = a1 * x_continuo + b1
15
16     a2, b2, c2 = ajuste_poli2(x, y)
17     y2_continua = a2 * x_continuo**2 + b2 * x_continuo + c2
18
19     a3, b3, c3, d3 = ajuste_poli3(x, y)
20     y3_continua = a3 * x_continuo**3 + b3 * x_continuo**2 + c3 * x_continuo + d3
21
22     a4, b4, c4, d4, e4 = ajuste_poli4(x, y)
23     y4_continua = a4 * x_continuo**4 + b4 * x_continuo**3 + c4 * x_continuo**2 + d4 * x_continuo + e4
24
25     ae, be = ajuste_exponencial(x, y)
26     y5_continua = ae * np.exp(be * x_continuo)
27
28     curvas_continuas = [y1_continua, y2_continua, y3_continua, y4_continua, y5_continua]
29
30     for i in range(5):
31         salvar_grafico(x, y, x_continuo, curvas_continuas[i], f'Ajuste {modelos[i]}', nomes_arquivos[i])
32
33
34     plt.figure(figsize=(10, 6))
35     plt.scatter(x, y, color='black', label='Dados observados')
36     for i in range(5):
37         plt.plot(x_continuo, curvas_continuas[i], label=f'{modelos[i]}')
38     plt.xlabel('Tempo (anos)')
39     plt.ylabel('População (milhares)')
40     plt.title('Ajustes das funções')
41     plt.legend()
42     plt.grid(True)
43     plt.tight_layout()
44     plt.savefig('todos_ajustes.png')
45     plt.close()
```

Fonte: Elaboração Própria

Nesta parte, foram gerados gráficos individuais para cada modelo, exibindo os pontos reais e a curva ajustada. Foi criada uma versão contínua de x, chamada `x_continuo`, para desenhar as curvas suavemente. Os gráficos são salvos como imagens `.png`, facilitando a análise visual de cada ajuste. Também é gerado um gráfico geral com todos os modelos sobrepostos.

Além disso, no próximo trecho são criados dois gráficos de barras: um para os erros médios quadráticos e outro para os valores de R^2 . Cada barra representa um modelo, e os valores numéricos são exibidos acima delas. Todos os gráficos foram criados utilizando a biblioteca `matplotlib`.

Trecho 8 - Gráficos dos EQM e R^2

```
1  # GRÁFICO DOS ERROS OBTIDOS
2  def grafico_erro(modelos, erros):
3      fig, ax = plt.subplots()
4      barras = ax.bar(modelos, erros, color='blue')
5      ax.bar_label(barras, fmt='%.2f', padding=3)
6
7      ax.set_title('Erro Médio Quadrático')
8      ax.set_ylabel('Erro')
9      ax.grid(True, axis='y')
10     plt.tight_layout()
11     plt.savefig('erros_mq.png')
12     plt.show()
13     plt.close()
14
15
16  # GRÁFICO PARA COEFICIENTES DE DETERMINAÇÃO
17  def graficos_r2(modelos, rqs):
18      fig, ax = plt.subplots()
19      barras = ax.bar(modelos, rqs, color='seagreen')
20      ax.bar_label(barras, fmt='%.4f', padding=3)
21
22      ax.set_ylim(0, 1.05)
23      ax.set_title('Coeficiente de Determinação  $R^2$ ')
24      ax.set_ylabel('R²')
25      ax.grid(True, axis='y')
26      plt.tight_layout()
27      plt.savefig('coef_r2.png')
28      plt.show()
29      plt.close()
```

Fonte: Elaboração Própria

2.6. Interface

Para pleno funcionamento do programa, na função main foi criada uma implementado um menu interativo em linha de comando, permitindo ao usuário calcular os ajustes e visualizar as equações, EQM e R^2 , e visualizar esses dados em formato de tabela. Além disso, também são gerados gráficos para comparações entre os métodos.

Trecho 9 - Implementação da Interface no terminal

```
1 def main():
2     os.system("cls")
3
4     x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
5     y = np.array([10, 12.5, 15.6, 19.5, 24.4, 30.5, 38.1, 47.6, 59.5, 74.4])
6     modelos = ['Linear', 'Polinomial 2º', 'Polinomial 3º', 'Polinomial 4º', 'Exponencial']
7
8     print("\n===== TABELA INICIAL DE DADOS =====")
9     dados = list(zip(x, y))
10    print(tabulate(dados, headers=["Tempo (anos)", "População (milhares)"], tablefmt="grid"))
11
12    while True:
13        print("\n===== TRABALHO 5 - AJUSTE DE CURVAS =====")
14        print("[1] Calcular ajustes")
15        print("[2] Comparar coeficientes de determinação R²")
16        print("[3] Comparar erros quadráticos médios")
17        print("[4] Gerar gráficos dos ajustes")
18        print("[5] Sair")
19        op = input("Opção: ")
20
21        if op == '1':
22            os.system("cls")
23            eqs, rqs, errs = calcular_modelos(x, y)
24            tabela = list(zip(modelos, eqs, rqs, errs))
25            print("\nEquações Ajustadas:")
26            print(tabulate(tabela, headers=["Modelo", "Equação", "R²", "Erro MQ"], tablefmt="grid", floatfmt=".4f"))
27
28        elif op == '2':
29            os.system("cls")
30            _, rqs, _ = calcular_modelos(x, y)
31            tabela = list(zip(modelos, rqs))
32            print("\nCoeficientes de Determinação R²:")
33            print(tabulate(tabela, headers=["Modelo", "R²"], tablefmt="grid", floatfmt=".4f"))
34            graficos_r2(modelos, rqs)
35
36        elif op == '3':
37            os.system("cls")
38            _, _, errs = calcular_modelos(x, y)
39            tabela = list(zip(modelos, errs))
40            print("\nErros Quadráticos Médios:")
41            print(tabulate(tabela, headers=["Modelo", "Erro MQ"], tablefmt="grid", floatfmt=".4f"))
42            grafico_erro(modelos, errs)
43
44        elif op == '4':
45            os.system("cls")
46            gerar_graficos_ajustes(x, y)
47            print("\nGráficos dos ajustes gerados e salvos com sucesso!")
48
49        elif op == '5':
50            print("Encerrando ... ")
51            break
52
53        else:
54            print("Opção inválida. Tente novamente.")
```

Fonte: Elaboração Própria

3. ANÁLISE E DISCUSSÃO

Neste tópico será realizada a comparação entre os métodos de ajuste de curvas utilizados: Ajuste Linear, Ajuste Polinomial (Graus 2, 3 e 4) e o Ajuste Exponencial. Os métodos foram aplicados sobre o mesmo conjunto de dados experimentais, permitindo uma avaliação consistente entre eles.

Tabela 1 - Conjunto de dados utilizados

Tempo (anos)	População (milhares)
0	10.0
1	12.5
2	15.6
3	19.5
4	24.4
5	30.5
6	38.1
7	47.6
8	59.5
9	74.4

Com base nesses dados, foram criadas as equações utilizando cada um dos métodos, além do R^2 e o EQM. Segue abaixo a tabela com os resultados obtidos.

Tabela 2 - Resultados obtidos

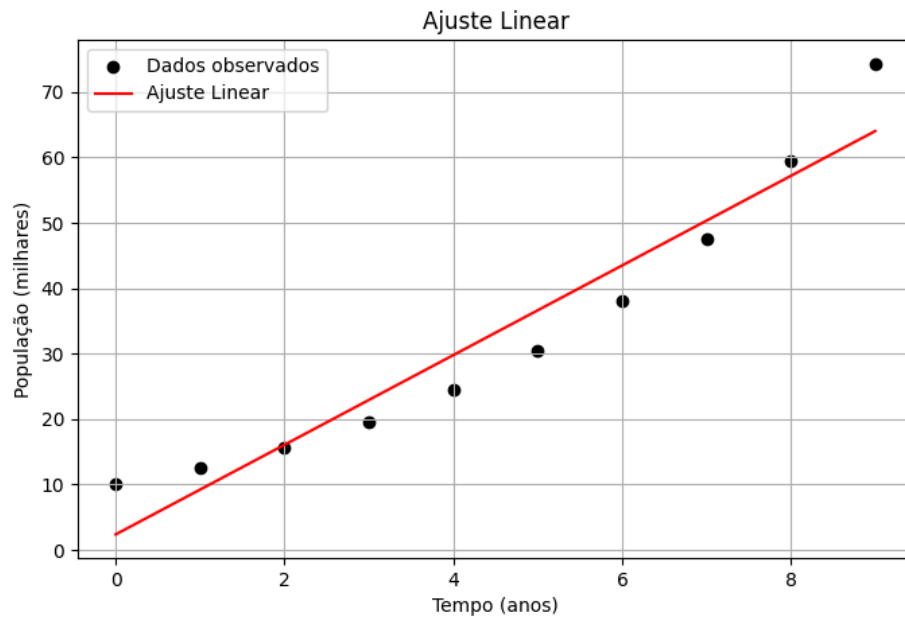
Modelo	Equação	R^2	EQM
Linear	$y = 6.8515x + 2.3782$	0.9289	5.4460
Polinomial 2º	$y = 0.7379x^2 + 0.2106x + 11.2327$	0.9978	0.9546
Polinomial 3º	$y = 0.0538x^3 + 0.0115x^2 + 2.6911x + 9.8768$	1.0000	0.1303
Polinomial 4º	$y = 0.0032x^4 + -0.0034x^3 + 0.3323x^2 + 2.1194x + 10.0140$	1.0000	0.0188
Exponencial	$y = 9.9962 * e^{(0.2230x)}$	1.0000	0.0132

Fonte: Elaboração Própria

Com base na tabela gerada, pode-se observar que o modelo exponencial apresentou o menor erro e o maior valor de R^2 , indicando o melhor desempenho. O modelo polinomial de grau 4 também teve boa precisão, seguido pelos polinomiais

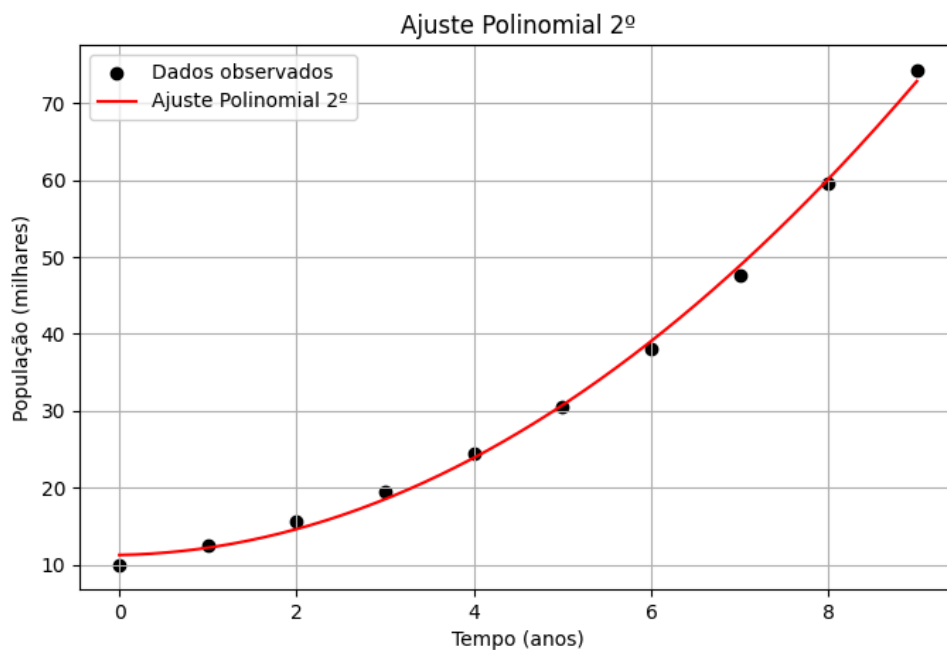
de graus 3 e 2. O ajuste linear foi o menos preciso, mas ainda assim coerente com os dados. Por fim, foram gerados os gráficos de todos os ajustes de curva para análise e também foram gerados os gráficos de comparação entre os valores de R^2 de cada ajuste, juntamente com os EQM.

Figura 1 - Ajuste Linear



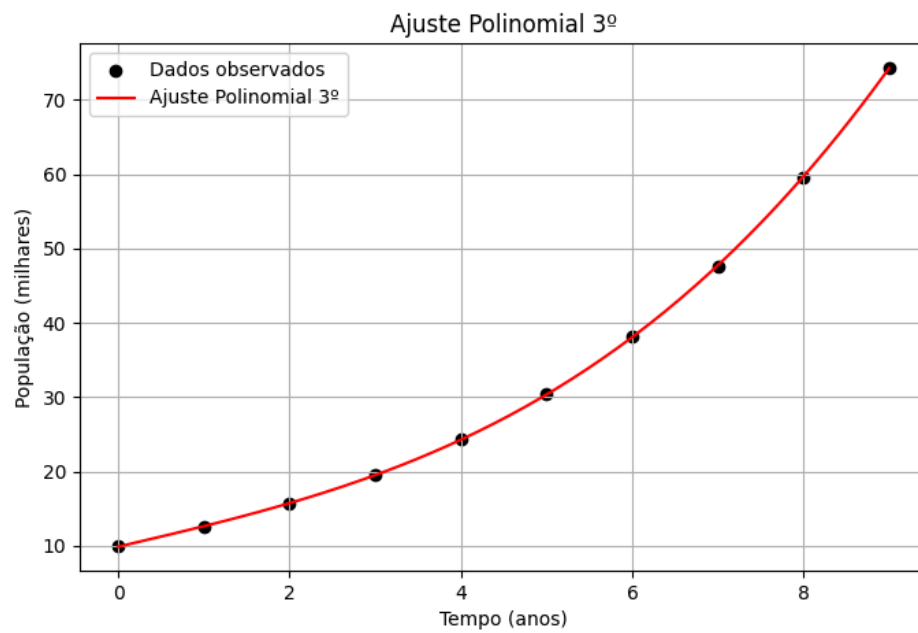
Fonte:Elaboração Própria utilizando Matplotlib

Figura 2 - Ajuste Polinomial de Grau 2



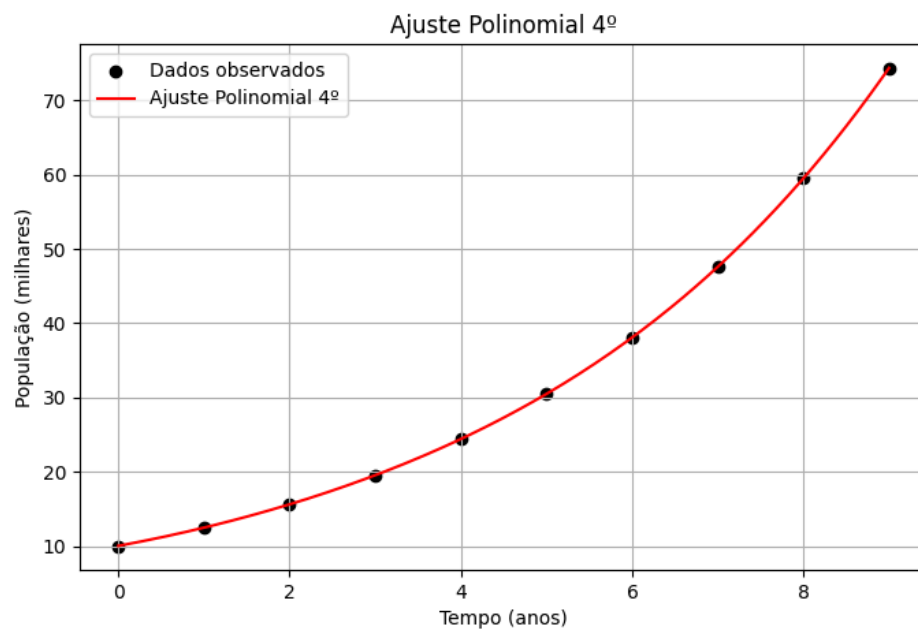
Fonte:Elaboração Própria utilizando Matplotlib

Figura 3 - Ajuste Polinomial de Grau 3



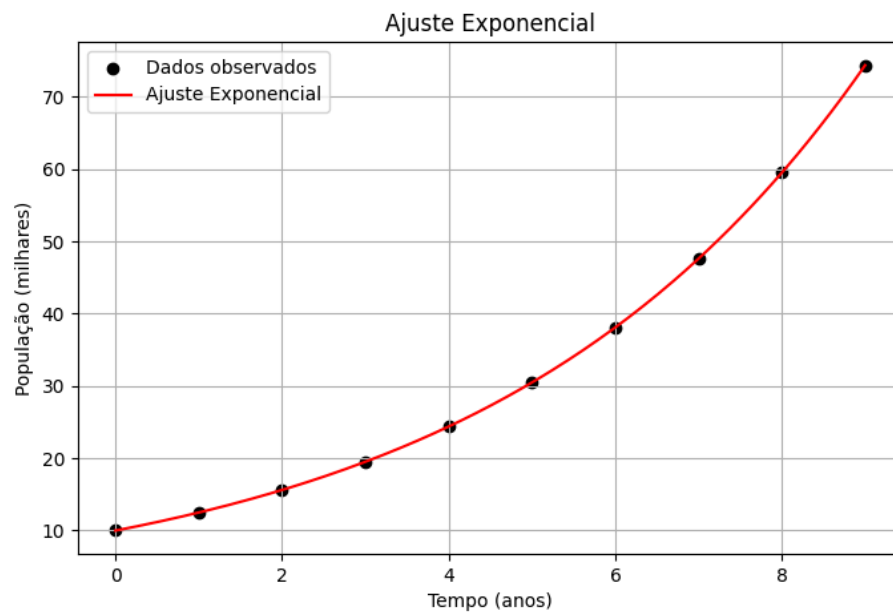
Fonte:Elaboração Própria utilizando Matplotlib

Figura 4 - Ajuste Polinomial de Grau 4



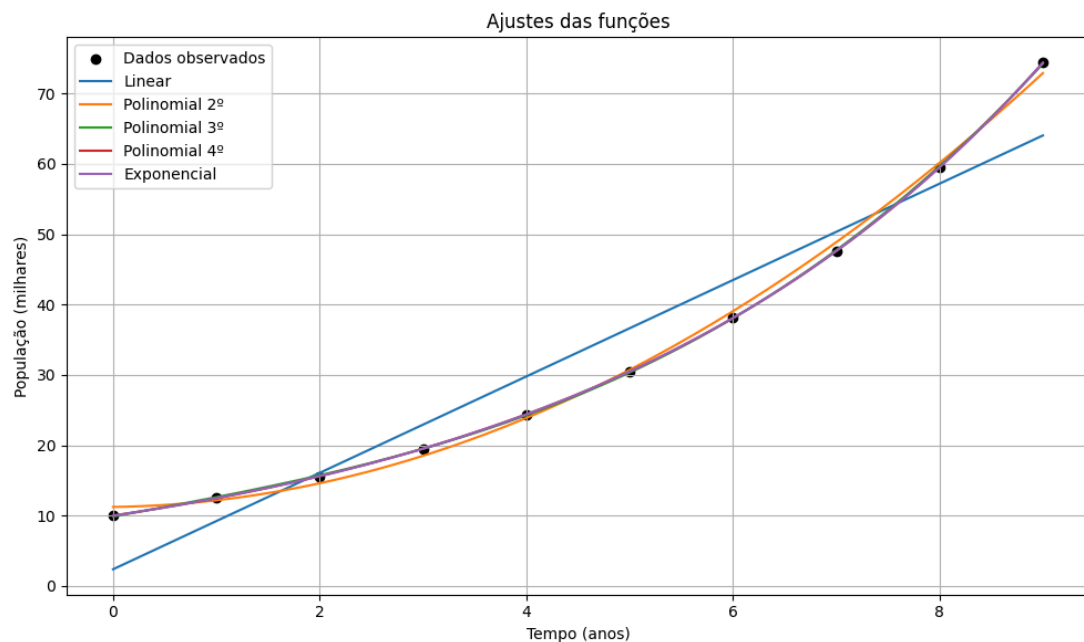
Fonte:Elaboração Própria utilizando Matplotlib

Figura 5 - Ajuste Exponencial



Fonte:Elaboração Própria utilizando Matplotlib

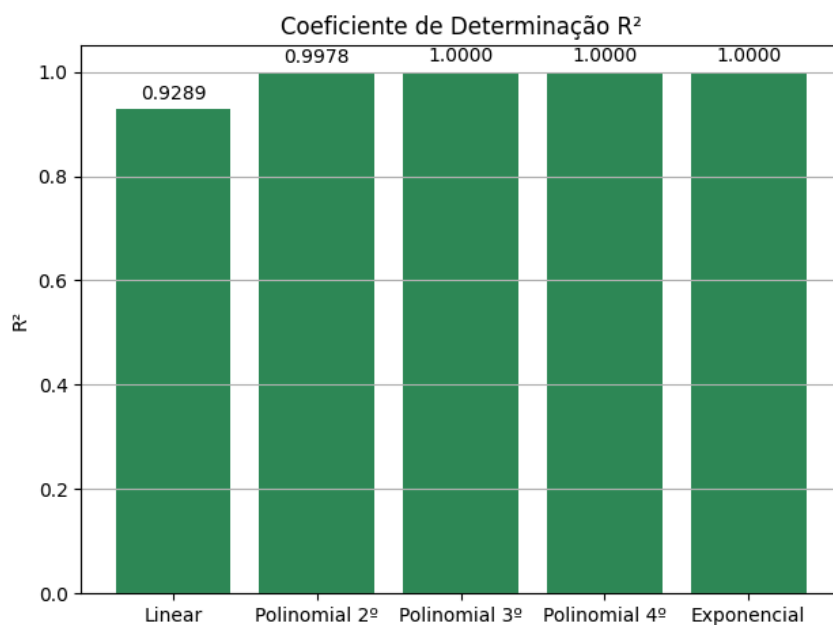
Figura 6 - Comparação dos Ajustes



Fonte:Elaboração Própria utilizando Matplotlib

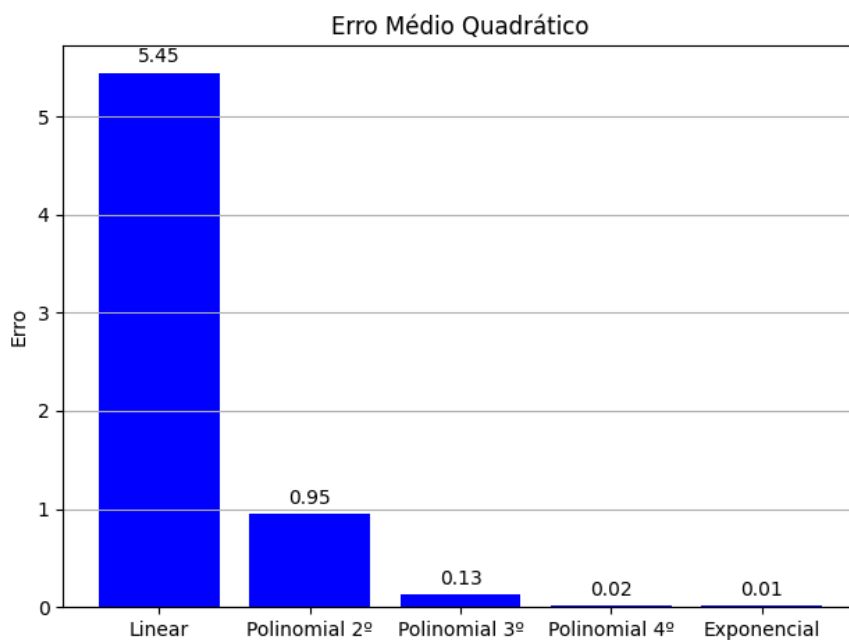
Além dos gráficos de todos os ajustes de curvas, também foram gerados os gráficos de barras de R^2 e EQM, para todos os tipos de ajustes de curvas:

Figura 7 - Comparação entre R^2



Fonte:Elaboração Própria utilizando Matplotlib

Figura 8 - Comparação entre EQM



Fonte: Elaboração Própria usando Matplotlib

Os gráficos mostram que os modelos de maior grau conseguem acompanhar melhor a curvatura dos dados. A curva do polinômio exponencial foi a que mais se aproximou dos pontos reais em todo o intervalo. Os gráficos de barras reforçaram isso, destacando a diferença entre os erros quadráticos médios e o R^2 dos métodos.

4. CONCLUSÃO

Neste trabalho, foram aplicados e comparados diferentes métodos de ajuste de curvas com base em dados de crescimento populacional, dados pelo enunciado do trabalho. Os resultados mostraram que todos os modelos apresentaram boa precisão, com valores de R^2 elevados.

O modelo exponencial foi o que melhor se ajustou aos dados, com o menor erro médio quadrático e o maior coeficiente de determinação. O polinomial de grau 4 também teve grande precisão com os dados reais, sendo uma boa alternativa para o modelo exponencial.

Portanto, conclui-se que a escolha do modelo depende do comportamento dos dados. Modelos mais complexos, como polinomiais de grau elevado ou exponenciais, podem representar melhor curvas acentuadas, enquanto modelos mais simples são mais indicados para dados com tendências mais lineares.